

Table of Contents

[DCDiffMeterDefTopic](#)

[DCInstrumentationDefTopic](#)

[dcdiffmeter_about.1.2](#)

[dcdiffmeter_about.1.3](#)

[dcdiffmeter_about.1.4](#)

[dcdiffmeter_about.1.5](#)

[dcdiffmeter_apps.4.01](#)

[dcdiffmeter_apps.4.02](#)

[dcdiffmeter_apps.4.03](#)

[dcdiffmeter_apps.4.04](#)

[dcdiffmeter_apps.4.05](#)

[dcdiffmeter_apps.4.06](#)

[dcdiffmeter_apps.4.07](#)

[dcdiffmeter_apps.4.08](#)

[dcdiffmeter_apps.4.09](#)

[dcdiffmeter_apps.4.10](#)

[dcdiffmeter_program.3.01](#)

[dcdiffmeter_program.3.02](#)

[dcdiffmeter_program.3.03](#)

[dcdiffmeter_program.3.04](#)

[dcdiffmeter_program.3.05](#)

[dcdiffmeter_program.3.06](#)

[dcdiffmeter_program.3.07](#)

[dcdiffmeter_program.3.08](#)

[dcdiffmeter_program.3.09](#)

[dcdiffmeter_program.3.10](#)

[dcdiffmeter_program.3.11](#)

[dcdiffmeter_program.3.12](#)

[dcdiffmeter_program.3.13](#)

[dcdiffmeter_program.3.14](#)
[dcdiffmeter_program.3.15](#)
[dcdiffmeter_program.3.16](#)
[dcdiffmeter_program.3.17](#)
[dcdiffmeter_too.2.02](#)
[dcdiffmeter_too.2.03](#)
[dcdiffmeter_too.2.04](#)
[dcdiffmeter_too.2.05](#)
[dcdiffmeter_too.2.06](#)
[dcdiffmeter_too.2.07](#)
[dcdiffmeter_too.2.08](#)
[dcdiffmeter_too.2.09](#)
[dcdiffmeter_too.2.10](#)
[dcdiffmeter_too.2.11](#)
[dcdiffmeter_too.2.12](#)
[dcdiffmeter_too.2.13](#)
[dcdiffmeter_too.2.14](#)
[dcdiffmeter_too.2.15](#)
[dcdiffmeter_too.2.16](#)
[dcdiffmeter_too.2.17](#)

DCDiffMeterDefTopic



About the DCDiffMeter

About the DCDiffMeter

The DC30 and DC75 DC Differential Voltmeter (DCDiffMeter) is a DC instrument that makes small voltage measurements on a large common-mode signal. In addition, each differential voltmeter has three lower-voltage ranges with improved specifications over the channel DCVI's meters. The DC30 and DC75 DCDiffMeter can also be used with the DCTime Measurement Units on the DC30 and DC75 instrument boards, through the asynchronous trigger lines, to make measurements.

The DC30 and DC75 instruments include two DC Differential Voltmeters.

The following topics are included:

- [Features](#)
- [Safety Information](#)
- [Key Terms](#)
- [DC30/75 DCDiffMeter Software Licensing Requirements](#)

DCDiffMeter information includes the following sections:

- [DCDiffMeter Theory of Operation](#)
- [DCDiffMeter Programming](#)
- [DCDiffMeter Examples](#)

Related Topics

- [DC Debug Displays](#)
- [DC30 Specifications](#)
- [DC75 Specifications](#)
- [DCDiffMeter](#) (VBT syntax)

- [DC30 DIB Design Guidelines](#)

- [DC75 DIB Design Guidelines](#)

For more information on DC30 and DC75 instruments, see:

- [DC30 Instrument Board](#)

- [DC75 Instrument Board](#)

- [About the DCTime](#)

- [About the DCVI](#)

For information on the UltraVI80 DCDiffMeter, see [About the UltraVI80 DCDiffMeter](#).

<

>

DCInstrumentationDefTopic

<

>

DCDiffMeter Theory of Operation

DCDiffMeter Theory of Operation

A DC30 or DC75 DC Differential Voltmeter (DCDiffMeter) is a differential meter that performs small voltage measurements on large common-mode signals. A DCDiffMeter has three lower voltage ranges with improved accuracy specifications over the DC30 and DC75 board DCVI meters. A DC30 or DC75 board has two DCDiffMeters.

A DCDiffMeter can be asynchronously triggered to perform a measurement by listening to one of the four onboard DC Time Measurement Unit (DCTime) trigger lines.

The high and low sides connect to the DUT using the channel sense lines. The sense lines that can be used depend on the instrument board and DCDiffMeter being connected.

The following topics are available:

- DCDiffMeter Overview
- DCDiffMeter Connections
- DCDiffMeter Alarms
- DCDiffMeter Errors

For information on DCDiffMeter instrument performance specifications and standards, see:

- Differential Voltmeter (DC30)
- Differential Voltmeter (DC75)

< >		
	2015 Teradyne, Inc. All rights reserved.	

dcdiffmeter_about.1.2

<

>

About the DCDiffMeter : Features

Features

The following is a list of DC30 and DC75 DCDiffMeter features:

DC Instrument Features		
Feature	DC30	DC75
DCDiffMeter	2	2
DGS	4	4
Max. differential voltage	5V	5V
Max. common mode input voltage	30V	75V
Voltage ranges	5V, 2.5V, 1V, 0.5V, 250mV, 100mV, 50mV	5V, 2.5V, 1V, 0.5V, 250mV, 100mV, 50mV
Capture memory depth	512 samples	512 samples
Meter ADC capture rates	10s to 1270s, in 10s increments	10s to 1270s, in 10s increments

For more information, see:

- DC30 Specifications
- DC75 Specifications

< >		
	2015 Teradyne, Inc. All rights reserved.	

dcdiffmeter_about.1.3

	
---	---

About the DCDiffMeter : Safety Information

Safety Information

Follow general safety procedures when operating DC30 and DC75 instrument boards. Read all system and instrument safety instructions before operating the test system. Only trained personnel should operate the test system. For more information, see [Test System Safety](#) in the [Safety](#) section.

Warning:

Even if a DCVI instrument is in a programmed state that is not forcing current or voltage, this does not imply any sense of safety. If the instrument is powered on, the potential of danger exists at the DIB.

Tester Cover

Keep the test head cover in place when the tester is in operation. If the test head cover is removed, an interlock is activated powering down the test system. Only trained and authorized personnel should operate the tester with the test head cover removed.

Warning:

Even with Gate Off, this does not imply any sense of safety. Voltage can exist at the channel output. Follow proper safety rules before handling a DIB. For proper handling of a DIB, see [About Operator and Safety Information](#) in the [Safety](#) section.

DIB Handling

Circuitry on a DIB can store hazardous potential depending on the components on the DIB (capacitors). Use proper care when handling a DIB even after the test system is powered down. Since DIBs are designed by customers, the test system cannot track if a DIB has safely dissipated stored energies.

Warning:

Be sure that any power storage devices on a DIB have discharged stored potential before handling the DIB or personnel injury can occur.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

dcdiffmeter_about.1.4

<

>

About the DCDiffMeter : Key Terms

Key Terms

capture	The act of a meter making measurements on a signal and saving the samples to memory.
Capture memory	The number of samples measured in a capture that can be stored. DC instrumentation can capture 512 samples.
DIB Access	Allows a connection to be made to a DUT pin, connected to one instrument, by routing the signal of the secondary instrument into the primary instrument’s DIB Access line and out the pin through the primary instrument’s signal path.
DGS	Device ground sense is a sense signal that tracks the voltage of ground on a DIB.
force line	Wiring that carries programmed output from the DCVI to a DUT pin.
guard	Shielding around the sense and force lines set at the output of the sense line to increase signal integrity and to protect from leakage.
sense line	High impedance wiring used to measure the output of the force line. Also used to make connections to the DCDiffMeter and DCTime to a DUT pin. This line is part of the control loop.
XOR	An output from the DCTime Time front end that goes out to the DIB through DIB access.

<	>	
	2015 Teradyne, Inc. All rights reserved.	

dcdiffmeter_about.1.5

<	>
-----------------	-----------------

[About the DCDiffMeter](#) : DC30/75 DCDiffMeter Software Licensing Requirements

[DC30/75 DCDiffMeter Software Licensing Requirements](#)
Operation of the DC30 and DC75 does not require a software license.
For more information, see [IG-XL Licensing](#) in IG-XL Administration help.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcdiffmeter_apps.4.01

<	>
-----------------	-----------------

DCDiffMeter Examples

DCDiffMeter Examples

This section provides device-based test program examples for the DC30 or DC75 DC Differential Voltmeter (DCDiffMeter).

The following topics are available:

- | | |
|---|--|
| • | DCDiffMeter Delta Voltage Measurement Test |
| • | DCDiffMeter VBT Examples |

For more information, see:

- | | |
|---|--|
| • | DCDiffMeter (VBT Syntax Reference) |
| • | DCDiffMeter Programming |

<	>
-----------------	-----------------

dcdiffmeter_apps.4.02

<	>
-----------------	-----------------

[DCDiffMeter Examples](#) : DCDiffMeter Delta Voltage Measurement Test

DCDiffMeter Delta Voltage Measurement Test

This test makes a differential measurement on the voltage regulator.
The required programming steps are:

- | | |
|---|-------------------------------------|
| • | Set Up the DUT |
| • | Set Up DataTool |
| • | Create the VBT Code |

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcdiffmeter_apps.4.03

<

>

DCDiffMeter Examples : DCDiffMeter Delta Voltage Measurement Test : Set Up the DUT

Set Up the DUT

The figure shows the DUT for this test.



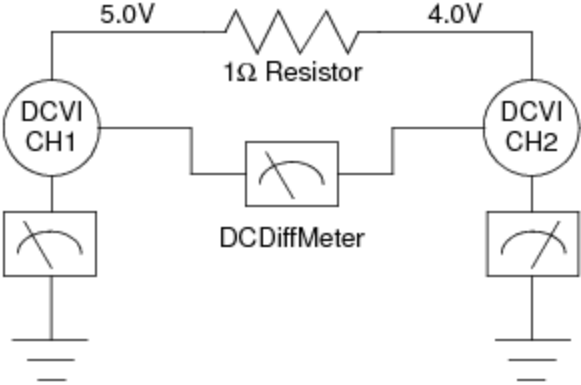
DUT Block Diagram

The table lists the specifications for this test.

DUT Specification Sheet

Pin Name	Pin Function
ch1	Connects to channel 1
ch2	Connects to channel 2

This figure shows DUT connections and DIB wiring required for this test.



DIB Layout

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcdiffmeter_apps.4.04

<

>

DCDiffMeter Examples : DCDiffMeter Delta Voltage Measurement Test : Set Up DataTool

Set Up DataTool

This test uses the following DataTool sheets and data:

- Pin Map Sheet
- Channel Map Sheet
- Test Instances Sheet
- Flow Table Sheet

Pin Map Sheet

Pin Map			
Group Name	Pin Name	Type	Comment
	ch1	Analog	
	ch2	Analog	
	ch1hi	Analog	
	ch1lo	Analog	

Channel Map Sheet

Channel Map				
DIB ID:		View Mode: Signal		
Device Under Test		Tester Channel		
Pin Name	Package Pin	Type	Site 0	Comment
ch1		DCVI	23.sense1	
ch2		DCVI	23.sense2	
ch1hi		DCDiffMeter	23.sense1	
ch1lo		DCDiffMeter	23.sense2	

Test Instances Sheet

Test Instances			
Test Name		Instance	
Type	Name	Called As	
DCDiffMeterMeasurement	VBT	DiffMeterMeasure	

Flow Table Sheet

Flow Table						
Gate			Command			
Label	Enable	Job	Part	Env	Opcode	Parameter
					Test	DCDiffMeterMeasurement

<

>

dcdiffmeter_apps.4.05

	
---	---

[DCDiffMeter Examples](#) : [DCDiffMeter Delta Voltage Measurement Test](#) : Create the VBT Code

Create the VBT Code

The following VBT code performs this test. Cut and paste it into the VBT editor.

Public Function DiffMeterMeasure()

' Program the DCVI connected to the Input pin

With TheHdw.DCVI.Pins("ch1")

.Connect

.Gate = False

.Mode = tIDCVIModeVoltage

.Voltage = 5

.VoltageRange.Autorange = True

.Current = 0.1

.CurrentRange.Autorange = True

End With

' Program the DCVI connected to the Output pin

With TheHdw.DCVI.Pins("ch2")

.Connect

.Gate = False

.Mode = tIDCVIModeVoltage

.Voltage = 5

.VoltageRange.Autorange = True

.Current = 0.1

.CurrentRange.Autorange = True

End With

' Gate on all DCVIs

TheHdw.DCVI.Pins("ch1,ch2").Connect

' Specify the low side of the DCDiffMeter

TheHdw.DCDiffMeter.Pins("ch1hi").LowSide.Pins = ("ch1lo")

' Set up the DCDiffMeter

With TheHdw.DCDiffMeter.Pins("ch1hi")

 .Connect tIDCDiffMeterConnectDefault

 .VoltageRange = 2.5

End With

' Program a wait time

TheHdw.Wait (0.001)

' Program the DCDiffMeter to make a measurement

Dim Results As New PinListData

Results = TheHdw.DCDiffMeter.Pins("ch1hi").Read(tlStrobe, 1, 100)

Call TheExec.Flow.TestLimit(resultVal:=Results, ForceResults:=tlForceFlow)

' Gate off and disconnect the DCVI

With TheHdw.DCVI.Pins("ch1,ch2")

 .Mode = tIDCVIModeVoltage

 .Gate = False

 .Disconnect

End With

End Function

dcdiffmeter_apps.4.06

<	>
-----------------	-----------------

[DCDiffMeter Examples](#) : DCDiffMeter VBT Examples

DCDiffMeter VBT Examples

The following VBT examples are included:

- | | |
|--|---|
| | <ul style="list-style-type: none">Specify Low Side of the DCDiffMeter |
| | <ul style="list-style-type: none">Set Up the DCDiffMeter |
| | <ul style="list-style-type: none">Program Individual Parameters to Program a DCDiffMeter PSet |
| | <ul style="list-style-type: none">Program Set Statements to Program a DCDiffMeter PSet |

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcdiffmeter_apps.4.07

<	>
-----------------	-----------------

[DCDiffMeter Examples](#) : [DCDiffMeter VBT Examples](#) : Specify Low Side of the DCDiffMeter

Specify Low Side of the DCDiffMeter

Public Function DiffMeterLowPin()

' Specify the low side of the DCDiffMeter

TheHdw.DCDiffMeter.Pins("ch1hi").LowSide.Pins = ("ch1lo")

End Function

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcdiffmeter_apps.4.08

<	>
-----------------	-----------------

[DCDiffMeter Examples](#) : [DCDiffMeter VBT Examples](#) : Set Up the DCDiffMeter

Set Up the DCDiffMeter

Public Function DiffMeterLowPin()

' Set up the DCDiffMeter

With TheHdw.DCDiffMeter.Pins("ch1hi")

 .Connect tIDCDiffMeterConnectDefault

 .VoltageRange = 2.5

End With

End Function

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcdiffmeter_apps.4.09

	
---	---

[DCDiffMeter Examples](#) : [DCDiffMeter VBT Examples](#) : Program Individual Parameters to Program a DCDiffMeter PSet

[Program Individual Parameters to Program a DCDiffMeter PSet](#)

[Public Function DCDiffMeterPSet\(\)](#)

[' Specify the low side of the DCDiffMeter](#)

TheHdw.DCDiffMeter.Pins("DCDiffHigh").LowSide.Pins = ("DCDiffLow")

[' Connect the DCDiffMeter to the DUT](#)

TheHdw.DCDiffMeter.Pins("DCDiffHigh").Connect

[' Add a PSet to the PSet collection](#)

TheHdw.DCDiffMeter.Pins("DCDiffHigh").PSets.Add ("MyPSet")

[' Get PSet from the PSet collection and program the PSet properties](#)

With TheHdw.DCDiffMeter.Pins("DCDiffHigh").PSets.Item("MyPSet")

.Capture.ExtraSamples = 3

.Capture.SampleRate = 10000

.Capture.SampleSize = 100

.VoltageRange = 5

End With

[' Apply PSet to hardware](#)

TheHdw.DCDiffMeter.Pins("DCDiffHigh").PSets.Item("MyPSet").Apply

[End Function](#)

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcdiffmeter_apps.4.10

<	>
-----------------	-----------------

[DCDiffMeter Examples](#) : [DCDiffMeter VBT Examples](#) : Program Set Statements to Program a DCDiffMeter PSet

Program Set Statements to Program a DCDiffMeter PSet

```
Public Function DCDiffMeterPSetSetStatement()  
' Specify the low side of the DCDiffMeter  
TheHdw.DCDiffMeter.Pins("DCDiffHigh").LowSide.Pins = ("DCDiffLow")  
' Connect the DCDiffMeter to the DUT  
TheHdw.DCDiffMeter.Pins("DCDiffHigh").Connect  
' Add a PSet to the PSet collection and program with a Set statement  
TheHdw.DCDiffMeter.Pins("DCDiffHigh").PSets.Add("MyPSet").Set 5, False, 10000, 100, 3  
' Apply PSet to hardware  
TheHdw.DCDiffMeter.Pins("DCDiffHigh").PSets.Item("MyPSet").Apply  
End Function
```

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcdiffmeter_program.3.01

<	>
-----------------	-----------------

DCDiffMeter Programming

DCDiffMeter Programming

This topic describes programming of a DC30 or DC75 DC Differential Voltmeter (DCDiffMeter). The following topics are available:

- [DCDiffMeter DataTool Programming](#)
- [Connecting a DCDiffMeter](#)
- [Onboard Asynchronous Triggering of a DCDiffMeter](#)
- [Using DCDiffMeter in the Mixed Signal Workshop](#)
- [Visual Basic for Test](#)
- [DCDiffMeter Reset Values](#)
- [DCDiffMeter Pattern Control](#)

For more information, see:

- [DCDiffMeter \(VBT Syntax Reference\)](#)
- [DCDiffMeter Examples](#)

< >		
	2015 Teradyne, Inc. All rights reserved.	

dcdiffmeter_program.3.02

<	>
-----------------	-----------------

[DCDiffMeter Programming](#) : DCDiffMeter DataTool Programming

DCDiffMeter DataTool Programming

This topic provides instructions on setting up parameters in worksheets specific to the DCDiffMeter. The following topics are included:

- [Programming the Pin Map](#)
- [Programming the Channel Map](#)

For more information on DataTool setup and programming, see [About DataTool](#).

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcdiffmeter_program.3.03

<	>
-----------------	-----------------

[DCDiffMeter Programming](#) : [DCDiffMeter DataTool Programming](#) : Programming the Pin Map

Programming the Pin Map

A pin map defines the pins of a device and the type of each pin. You can enter any pin type for a DCDiffMeter. The recommended selection is:

Analog	This type is used for the DCDiffMeter.
--------	--

For more information, see [Pin Map Sheet](#) in the DataTool section.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcdiffmeter_program.3.04

<

>

DCDiffMeter Programming : DCDiffMeter DataTool Programming : Programming the Channel Map

Programming the Channel Map

A DC30/75 instrument board can support 250 sites. For each pin that is connected to a DCDiffMeter, an entry is required in the PinName, Type and each Site column. Type selection for DCDiffMeter instrument channels is:

DCDiffMeter	Sense line DIB address connection to a DCDiffMeter. In a channel map, both the High Side and the Low Side must have DCDiffMeter defined as the channel type.
-------------	--

For more information, see Channel Map Sheet in the DataTool section.

☒ **Note:**

You must specify both the high and low side in the channel map.

Site Column Entries for DC30

In the Site columns, the channel map is programmed with the DIB connector number, the sense line and optionally the DGS connection. The DIB connector location is dependent on tester configuration. The following table shows examples of valid entries:

DC30 Channel Map Entries			
Pin Name	Type	Site	Comment
HighSide	DCDiffMeter	23.sense1	Channel 1 at DIB slot 23.
LowSide1	DCDiffMeter	23.sense2	Channel 2 at DIB slot 23; use for assigning low side to a sense line.

LowSide2	DCDiffMeter	23.dgs1	Use for assigning low side to a DGS line or for a single-ended measurement referenced to the specified DGS.
----------	-------------	---------	---

You can see this information in the file <installdir>\tester\CurrentChannelMap.txt.

Site Column Entries for DC75

The DC75 supports internal multiplexed channels allowing more connections to resources without requiring relays on the DIB. Select multiplexed channels by entering the channel DIB address for the mux to which you wish to connect. The table lists the channel DIB addresses for multiplexed channels.

Channel DIB Address for Multiplexed Channels

DC75 Channel	A Mux	B Mux
Channel 1	1.sense1a	1.sense1b
Channel 2	1.sense2a	1.sense1b
Channel 3	1.sense3a	1.sense1b
Channel 4	1.sense4a	1.sense1b

In the Site columns, the channel map is programmed with the DIB connector number, the sense line with the mux side indicator and optionally the DGS connection. The DIB connector location is dependent on tester configuration. The following table shows examples of valid entries:

DC75 Channel Map Entries

Pin Name	Type	Site 0	Comment
HighSide	DCDiffMeter	50.sense1b	Channel 1b at DIB slot 50.
LowSide1	DCDiffMeter	50.sense2a	Channel 2a at DIB slot 50; use for assigning low side to a sense line.
LowSide2	DCDiffMeter	50.dgs1	Use for assigning low side to a DGS line or for a single-ended measurement referenced to the specified DGS.

You can see this information in the file <installdir>\tester\CurrentChannelMap.txt.

To have multiplexed connections, proper DIB layout is required. See [DC75 Multiplexed Channel Mapping](#) in the DIB Design Guidelines.

 Note:

A and B sides cannot be used at the same time during program execution.

[Channel Map Setup for DIB Access](#)

Program a channel map sheet for a DCDiffMeter as you would program the sheet for a DCVI channel. For more information, see [Channel Map Setup for DIB Access](#).

			
	2015 Teradyne, Inc. All rights reserved.		

dcdiffmeter_program.3.05

<	>
-----------------	-----------------

[DCDiffMeter Programming](#) : Connecting a DCDiffMeter

Connecting a DCDiffMeter

A DCDiffMeter has both a high and low side that are required to be connected. Connections to the DUT are made through the channel sense lines, or the low side can be connected to a DGS. The connections are described in the following topics:

- [Connecting a DCDiffMeter High and Low Side to a Sense Line](#)
- [Connecting a DCDiffMeter Low Side to DGS](#)

For more information, see:

- [DCDiffMeter Connections](#)
- [DCDiffMeter DGS \(DC30\)](#)
- [DCDiffMeter DGS \(DC75\)](#)

<	>
-----------------	-----------------

dcdiffmeter_program.3.06

<

>

[DCDiffMeter Programming](#) : [Connecting a DCDiffMeter](#) : Connecting a DCDiffMeter High and Low Side to a Sense Line

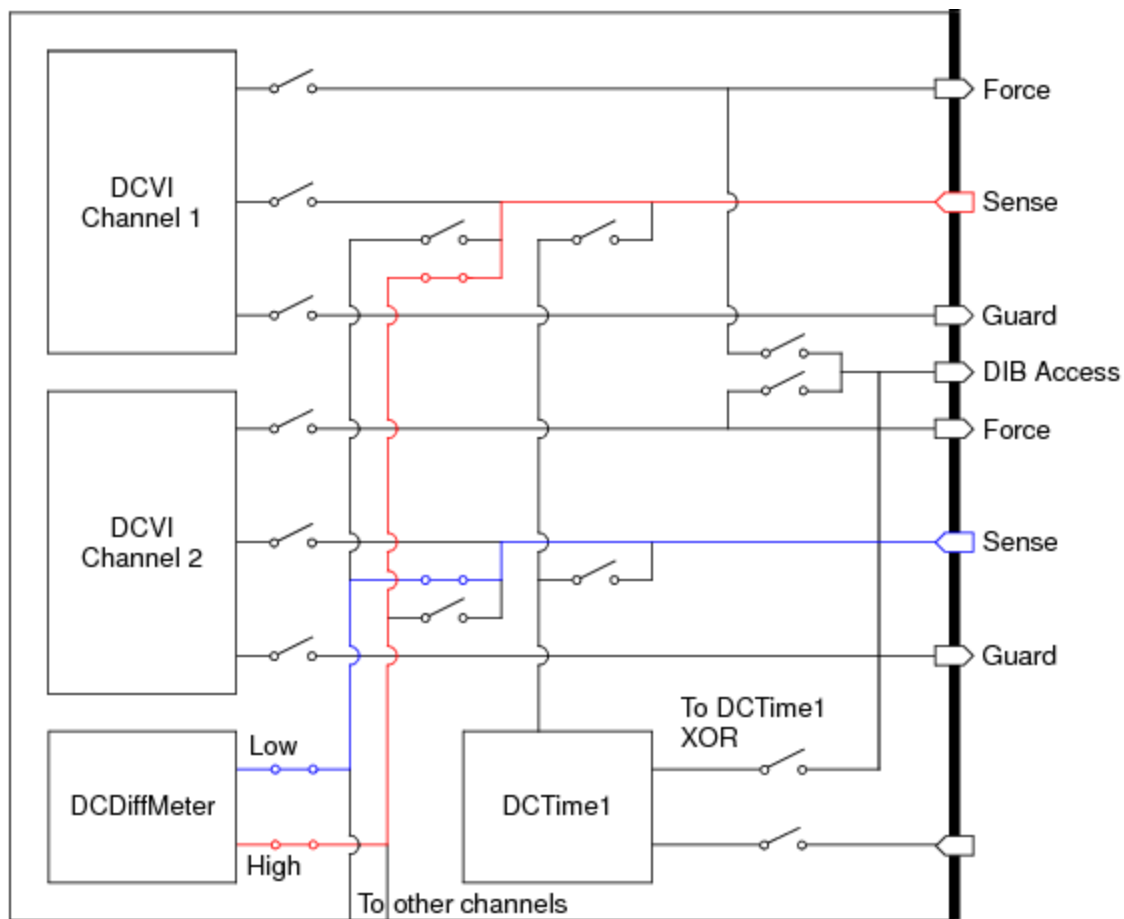
[Connecting a DCDiffMeter High and Low Side to a Sense Line](#)
The channel map requires entries similar to the following:

DCDiffMeter Example Channel Map				
Device Under Test		Tester Channel		
Pin Name	Package Pin	Type	Site 0	Comment
PinHigh		DCDiffMeter	1.sense1	Channel 1
PinLow		DCDiffMeter	1.sense2	Channel 2

Use the following VBT language to make connections to the DUT:

```
' Declare the lowside pin using the following statement
TheHdw.DCDiffMeter.Pins("PinHigh").LowSide.Pins = "PinLow"
' Connect both pins to the DUT using the following statement
TheHdw.DCDiffMeter.Pins("PinHigh").Connect tlDCDiffMeterConnectDefault
```

The figure shows the connections made:



DC30 DCDiffMeter Connections to Channel Sense Lines

< >	
2015 Teradyne, Inc. All rights reserved.	

dcdiffmeter_program.3.07

<

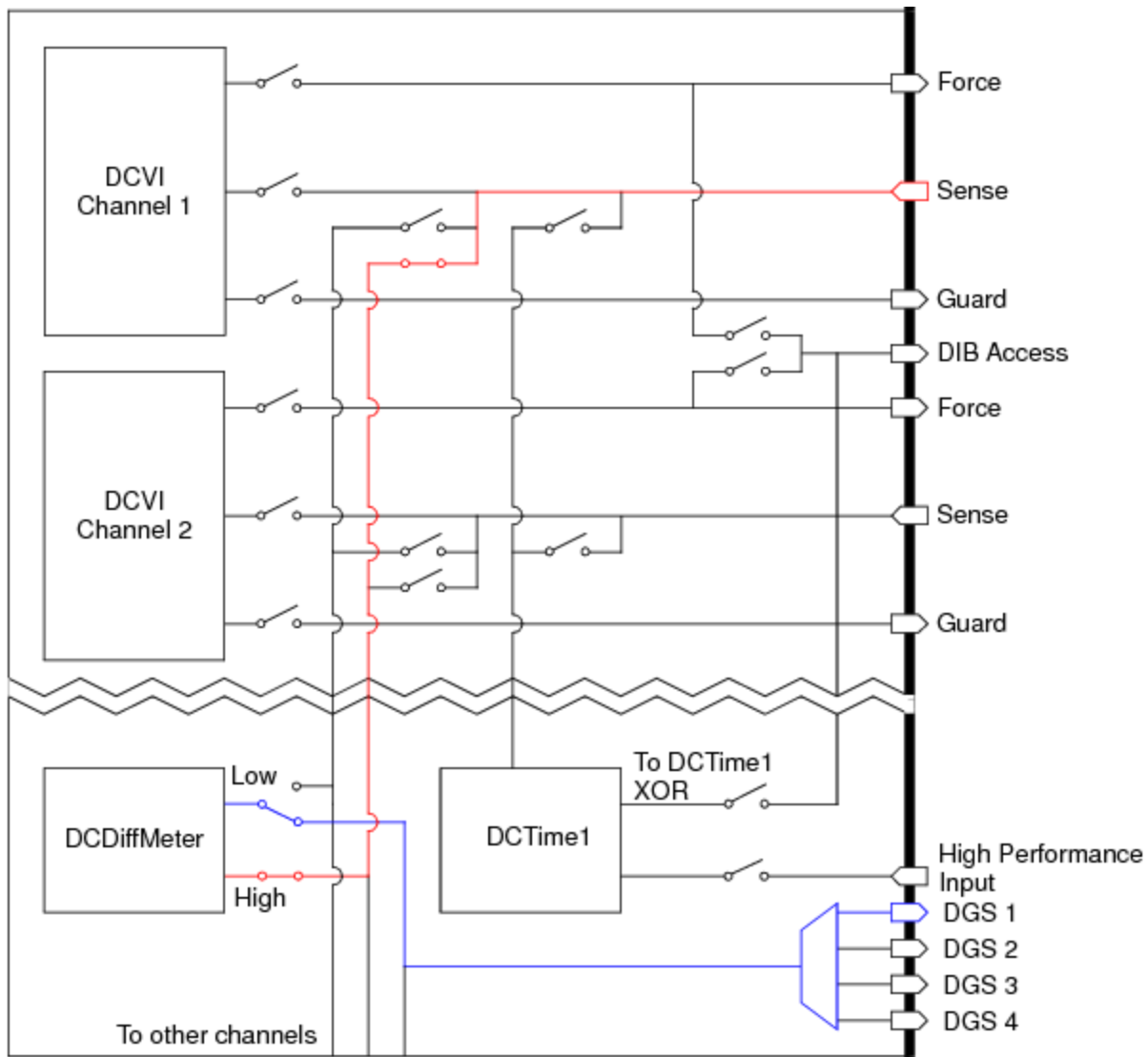
>

[DCDiffMeter Programming](#) : [Connecting a DCDiffMeter](#) : Connecting a DCDiffMeter Low Side to DGS

[Connecting a DCDiffMeter Low Side to DGS](#)
In some tests, you may want to connect the low side of the DCDiffMeter to the channel DGS. To do this the channel map requires entries similar to the following:

DCDiffMeter Example Channel Map				
Device Under Test		Tester Channel		
Pin Name	Package	Type	Site 0	Comment
PinHigh		DCDiffMeter	1.sense1	
PinLowDGS		DCDiffMeter	1.dgs1	

Use the following VBT language to make connections to the DUT:
' Declare the Lowside pin using the following statement
TheHdw.DCDiffMeter.Pins("PinHigh").LowSide.Pins = "PinLowDGS"
' Connect both pins using the following statement. PinHigh is connected to the
' DUT through the sense line, and PinLowDGS is connected the channel DGS.
TheHdw.DCDiffMeter.Pins("PinHigh").Connect tIDCDiffMeterConnectDefault
The figure shows the connections made:



DC30 DCDiffMeter Low Side Connection to DGS

<	>
2015 Teradyne, Inc. All rights reserved.	

dcdiffmeter_program.3.08

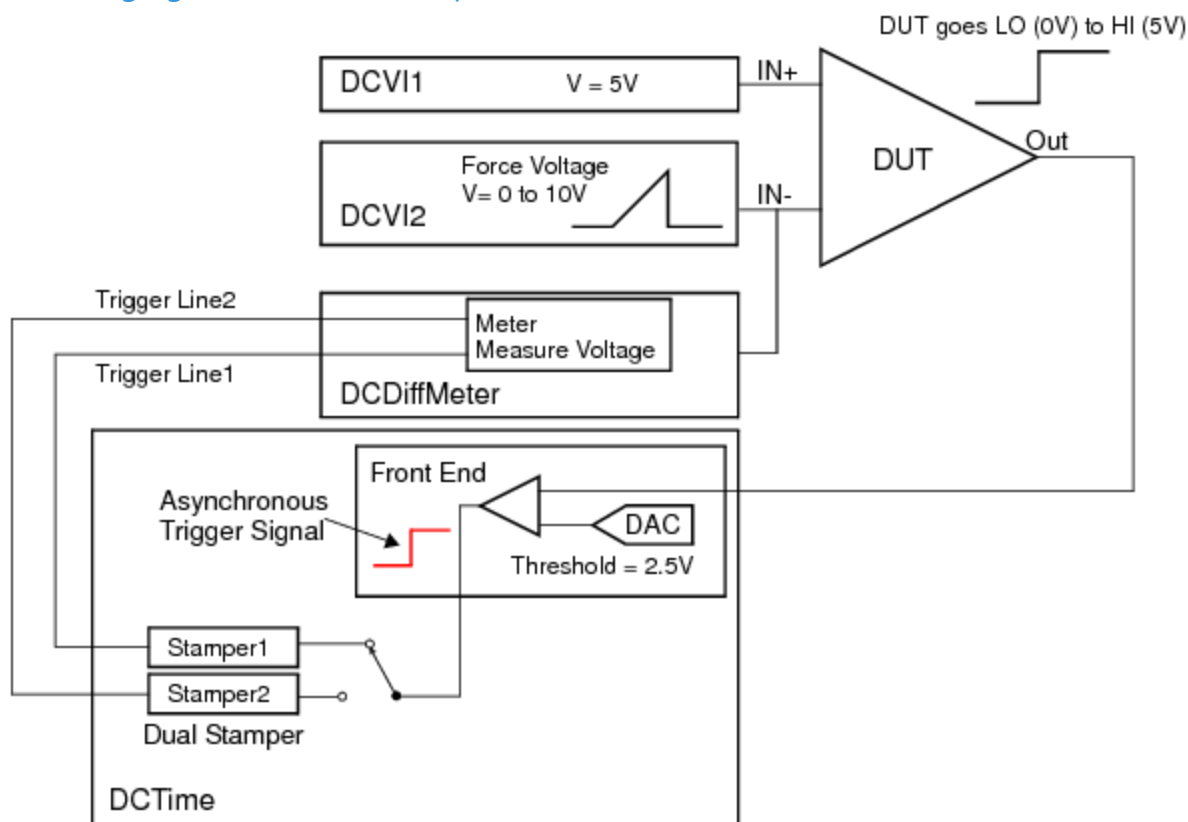


DCDiffMeter Programming : Onboard Asynchronous Triggering of a DCDiffMeter

Onboard Asynchronous Triggering of a DCDiffMeter

An asynchronous trigger allows you to trigger a DCDiffMeter (or a DCVI meter) on the occurrence of an event using a DCTime instrument. The DC30 and DC75 each have four asynchronous trigger lines, two per DCTime, that a DCDiffMeter can listen to.

Asynchronous triggering allows you to measure the state change of a device pin. A common example is a threshold test. One device pin receives a voltage ramp from a DCVI. When a certain threshold is crossed, the state of another pin changes. This pin is connected to a DCTime. The following figure shows an example.



Comparator Threshold Test Using a DCDiffMeter

The voltage range, voltage level, and input impedance of the DCTime are programmed per device specifications, and the mode is set to onboard trigger. The DCDiffMeter listens to the trigger line that the DCTime stamper drives when the event occurs.

When the threshold is crossed on the pin, a signal is output from the DCTime which in turn triggers the meter. The measurement made determines the voltage level the ramp was at when the change state occurred.

Making an asynchronous trigger measurement requires both the DCTime front end comparator and a stamper. The front end comparator outputs a signal to the stamper that places a signal onto a trigger line connected to the specified DCDiffMeter.

☑ Note:

A DCTime asynchronous trigger line can only connect to instruments on the same DC instrument board.

For more information, see:

- [Programming Asynchronous Trigger Measurements](#)

- [Asynchronous Trigger](#)

- [Pins.AsynchronousTrigger](#)

- [Onboard Asynchronous Triggering of a DCVI Meter](#)

Select the Trigger Pin

You must specify the pin that triggers a DCDiffMeter to make a measurement. The following VBT syntax selects a DCTime as the asynchronous trigger.

```
TheHdw.DCDiffMeter.Pins("MyPin").AsynchronousTrigger.SetInput "TrigPin" _
    , "DCTime", 1
```

Here, TrigPin is the name of the pin that is used to asynchronously trigger the meter. DCTime is the channel type for the pin TrigPin, and 1 indicates which trigger line is used from the specified DCTime.

Set the Trigger Pin Slope

You must program the slope of the trigger signal to the edge that a DCDiffMeter will trigger on.

The following syntax programs the slope for the meter.

```
TheHdw.DCDiffMeter.Pins("RampSourcePin").AsynchronousTrigger.Slope _
    = tISlopePositive
```

Select the Trigger Method

An asynchronous trigger can make a single measurement on the first state change only or can be set to automatically re-enable capturing data each time an event occurs.

To program a DCDiffMeter to be triggered only once (single trigger), use this syntax:

```
TheHdw.DCDiffMeter.Pins("MyPin").AsynchronousTrigger.Continuous = False
```

To program a DCDiffMeter to be triggered each time a trigger event occurs (continuous trigger), use this syntax:

```
TheHdw.DCDiffMeter.Pins("MyPin").AsynchronousTrigger.Continuous = True
```

Programming a Trigger Pin

The DCTime listens for an event. On receiving that event, it sends a signal through the trigger line to the instrument programmed to listen to it. The following are required to set up the DCTime:

- **OnBoard Trigger:** This mode notifies the DCTime to send a signal to another instrument using a trigger line. The DCTime listens on its inputs for an event to cross its threshold. Once this occurs, it sends a trigger signal to the instrument programmed to listen to the trigger line. Set this using the following syntax:

```
TheHdw.DCTime.Pins("MyPin").Mode = tlDCTimeModeOnBoardTrigger
```

- **Enabling the Trigger Pin:** The DCDiffMeter must be enabled to make an asynchronous measurement. Enable the trigger pin using the following syntax.

```
TheHdw.DCDiffMeter.Pins("MyPin").AsynchronousTrigger.Enable = True
```

You need not set the trigger pin slope for DCTime.

Other programming required for the DCTime depends on the test setup. For a programming example, see DCVI Meter Asynchronous Trigger Test Example.

Interaction with the Debug Display

The DCDiffMeter debug display has controls for an asynchronous trigger. These controls allow you to set up parameters for an asynchronous trigger, directly aligning with the VBT language noted above for the same purpose.

Normally, the debug display controls reflect the state of the instrument as set up by this syntax.

However, the Async Enable control is an exception. This control may be cleared if the Asynchronous Trigger is set for a single trigger event (that is, if the Async Continuous control is cleared) and the following test program sequence has occurred:

1. The asynchronous trigger is enabled.
2. A capture has been triggered or the DCDiffMeter has been strobed.

3.

The specified asynchronous trigger event occurs on the DCTime.

When the single trigger event occurs, therefore, the asynchronous trigger is no longer enabled. At this point:

- A call in VBT to .AsynchronousTrigger.Enable will return False.
- The Async Enable control on the DCDiffMeter debug display will be cleared.

For more information, see [DCDiffMeter Debug Display](#).

<

>

2015 Teradyne, Inc. All rights reserved.

dcdiffmeter_program.3.09

<	>
-----------------	-----------------

[DCDiffMeter Programming](#) : Using DCDiffMeter in the Mixed Signal Workshop

Using DCDiffMeter in the Mixed Signal Workshop

The DCDiffMeter can be used in the Mixed Signal Workshop as a capture instrument. When a DCDiffMeter capture is added to the MSW, the following field appears in the Capture Instrument Block:

Type	This menu allows selection of the type of test this capture instrument block is programmed for. For DC30 and DC75 select DCDiffMeter5Timing. This determines which instrument capabilities are checked against.
------	---

For more information, see [Mixed Signal Workshop](#).

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcdiffmeter_program.3.10

<	>
-----------------	-----------------

[DCDiffMeter Programming](#) : Visual Basic for Test

Visual Basic for Test

You can create DC instrumentation programs using VBT language in the VBT editor. Each instrument has its own properties and methods used for controlling the functionality of the instrument.

For more information, see [DCDiffMeter](#) in the VBT Syntax Reference.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcdiffmeter_program.3.11

<

>

DCDiffMeter Programming : DCDiffMeter Reset Values

DCDiffMeter Reset Values

Resetting the connections disconnects the DCDiffMeter from the DUT. Resetting the settings clears the DCDiffMeter memory and returns the DCDiffMeter to the default settings.

Perform a reset using the following VBT syntax:

TheHdw.DCDiffMeter.Pins(PinList).Reset(tlResetConnections Or tlResetSettings)

The table lists the reset state of DCDiffMeter parameters after a reset occurs.

DCDiffMeter Parameter Reset Values

DCDiffMeter Parameter	Reset Value
Asynchronous Trigger	
Slope	Positive
Channel Type	DCTime
Which	1
Enable	False
Continuous	False
Capture	
Extra samples	0
Sample rate	100kHz
Sample size	1
Filter	Bypass

Meter	
No Strobe	False
Sample rate	100kHz
Sample size	1
Voltage range	5.0V

<

>

dcdiffmeter_program.3.12

<	>
-----------------	-----------------

[DCDiffMeter Programming](#) : DCDiffMeter Pattern Control

DCDiffMeter Pattern Control

A pattern is used to precisely program an instrument at the vector level. You can program DCDiffMeter parameters using microcode or by applying PSets.

You create a pattern independently of the test program using the [PatternTool](#) or a text editor. Patterns are composed of a series of numbered digital vectors. These digital vectors define the data for each channel listed in the pattern, as well the pattern microcode used for controlling pattern execution.

The two forms of pattern control are:

Microcode	A microcode controls a single function, such as strobing a DCDiffMeter.
PSets	A PSet defines a programmed state for an instrument, so that a DCDiffMeter PSet contains parameters such as voltage range. When an instrument receives a PSet from a pattern, it enters the programmed state represented by that PSet.

The following topics are available.

- | | |
|---|--|
| • | Programming Patterns Using a Text Editor |
|---|--|
- | | |
|---|---|
| • | Retrieving Strokes and Capture Data |
|---|---|
- | | |
|---|---|
| • | Optimizing Pattern Restart Time |
|---|---|

•	DCDiffMeter Pattern Microcodes
•	PSets

[Pattern Compiler](#)

The [pattern compiler](#) takes a [text](#) .atp file and compiles it into a binary .pat file. This file can be edited using PatternTool and is the format accepted by the pattern loader.

[Pattern Reverse Compiler](#)

The [pattern reverse compiler](#) takes a [binary](#) .pat file and compiles it into a text .atp file. This file can be edited using a text editor.

[PatternTool](#)

PatternTool allows you to read and edit binary pattern files. Using PatternTool, you can display a compiled pattern file in easily readable, column formats. You can also directly edit a pattern file, instead of having to edit the ASCII source file and recompile it. You can then update the tester with the edited pattern file.

PatternTool is useful during debugging, specifically in debugging pattern files. PatternTool can also display HRAM (History RAM) data, as captured on the tester.

<	>	
	2015 Teradyne, Inc. All rights reserved.	

dcdiffmeter_program.3.13

	
---	---

[DCDiffMeter Programming : DCDiffMeter Pattern Control](#) : Programming Patterns Using a Text Editor

Programming Patterns Using a Text Editor

You can create and edit a pattern using a text editor. This form (.atp file) must be compiled into a .pat file before it can be loaded.

Instruments Statement

To use a DCDiffMeter from within a pattern, you must add the pin to the instruments statement, which appears at the top of the file:

```
instruments = {
    MyPin:DCDiffMeter;
}
```

For more information, see [Instruments Statement](#) in the Pattern Language section.

Vector Statements

In a vector statement, program a DCDiffMeter pin like other instruments that use microcodes:

```
(MyPin = Strobe) > TSet ;
```

If a pin has multiple resources behind it, then the statement appears a bit differently, with the instrument type included:

```
(MyPin:DCDiffMeter = Strobe) > TSet ;
```

PatternTool is also used to edit patterns containing DC instruments.

Pattern Example

The following is an example of a pattern in .atp form created in a text editor for the DCDiffMeter.

```
import tset ts1;
instruments = {
    MyPin:DCDiffMeter 1;
}
```

```
srm_vector MyPattern ( $tset, MyDigitalPin)
{
(MyPin:DCDiffMeter = Filter_On)
    > ts1  0      ;
    > ts1  0      ;
(MyPin:DCDiffMeter = Strobe)
    > ts1  0      ;
repeat 1000    > ts1  0      ;
    > ts1  0      ;
(MyPin:DCDiffMeter = PSet MyPSet)
    > ts1  0      ;
    > ts1  0      ;
halt          > ts1  0      ;
}
```

The figure shows the .atp file compiled using the Pattern Compiler and loaded into PatternTool.

TSet	Command	Label	Vector/Cycle	MyPin (DCDiffMeter)	MyDigitalPin	Comment
ts1			0	Filter_On	0	
ts1			1		0	
ts1			2	Strobe	0	
ts1			3		0	
ts1	repeat 1000		4		0	
ts1			5	PSet MyPSet	0	
ts1			6		0	
ts1	halt		7		0	

Signal Names

You must define all signal names that appear in your pattern, and you must associate all signal names with a signal.

You do not need to define trigger names before you use them. You can retrieve the samples using VBT by referring to the name of the capture signal used in the pattern.

Public Function MyTest() As Long

' Get captured data

Dim data_captured As New PinListData

Dim data_wave As New DSPWave

data_captured = TheHdw.DCDiffMeter.Pins("MyPin").Capture _
 .Waveforms("MySignal").Item.DSPWave

' Site Loop to retrieve results for both sites

For Each Site In TheExec.Sites

data_wave = data_captured

data_wave.CalcMean ' or other DSP processing

Next Site

End Function

<		>	
	2015 Teradyne, Inc. All rights reserved.		

dcdiffmeter_program.3.14

	
---	---

DCDiffMeter Programming : DCDiffMeter Pattern Control : Retrieving Strokes and Capture Data

Retrieving Strokes and Capture Data

When a meter is strobed, it stores a single measurement in meter memory. No name is associated with a strobe. Captures can consist of multiple measurements and are assigned a name. You define the characteristics of the capture, such as rate and number of samples, using a PSet preceding the Trig <name> microcode.

Use the following VBT syntax to retrieve a measurement strobed in a pattern:

Dim VMeasure As New PinListData

' Program the DCDiffMeter meter to make a strobe

```
VMeasure = TheHdw.DCDiffMeter.Pins("MyPin").Read(tlStrobe, 1, 10000)
```

' Program limit options and datalog the results

' Compare results to limits in Flow Table

Call TheExec.Flow.TestLimit(resultVal:=VMeasure, ForceResults:=tlForceFlow)

Strokes and captures are handled differently when reading back. Strokes are available in a FIFO format directly from the DCDiffMeter drivers, using a ReadMeterNoStrobe() function call. DSP must be used to retrieve capture data. For multiple captures, however, the captures are also retrieved in a FIFO format.

Note:

Although a test can be programmed to execute the strobe microcode at a 1s rate, exceeding the minimum capture rate of 10s/sample results in an ADC return value of zero.

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcdiffmeter_program.3.15

<	>
-----------------	-----------------

DCDiffMeter Programming : DCDiffMeter Pattern Control : Optimizing Pattern Restart Time

Optimizing Pattern Restart Time

IG-XL can speed up restarts of patterns that contain DC instruments. If you have run a pattern and you do not use VBT to change any parameters that affect the pattern before the next start of the same pattern, enabling optimization allows the pattern to restart more quickly. Improvement can range from 20% to 50%. The first run of a pattern always requires the full amount of time to start. After you have run a pattern once, subsequent restarts (including starts from a different label in the same pattern) benefit from the reduced startup time.

To achieve this restart efficiency, the following parameters must remain constant from one start to the next:

- Active site
- All DC PSet parameters (if you reprogram a PSet, do not apply it to hardware before you finish running the pattern)
- Source signal loaded in instrument memory

Note that faster pattern restarts may require adjustments to test program timing.

To enable pattern restart optimization, set the `PatternRestartOptimizationEnabled` property to `True`.

Example

The following example runs the pattern three times. The first run, starting at label `t0`, requires the full startup time. The second run, a restart, and the third run, a start at a different label in the pattern, are both optimized.

```
***** Start at label t0 *****
```

```
If TheExec.Sites.ActiveCount > 0 Then
```

```
TheHdw.Digital.Patterns.Pat("regulator").Start ("t0")
End If
TheHdw.Digital.Patgen.HaltWait
'***** Do something that does not change any DC instrument settings *****

'***** Restart at label t0 *****
If TheExec.Sites.ActiveCount > 0 Then
    TheHdw.Digital.Patgen.Restart
End If
TheHdw.Digital.Patgen.HaltWait
'***** Start at label t1 *****
If TheExec.Sites.ActiveCount > 0 Then
    TheHdw.Digital.Patterns.Pat("regulator").Start ("t1")
End If
TheHdw.Wait (0.01)
TheHdw.Digital.Patgen.HaltWait
```


dcdiffmeter_program.3.16

<

>

DCDiffMeter Programming : DCDiffMeter Pattern Control : DCDiffMeter Pattern Microcodes

DCDiffMeter Pattern Microcodes

DCDiffMeter pattern microcodes are instructions that you can place on an individual vector of a digital pattern. For that vector, a microcode instructs the DCDiffMeter to perform a particular action. A microcode controls a single function such as strobing the DCDiffMeter or enabling or disabling an alarm.

The format for specifying a DCDiffMeter microcode on a vector is either of the following:

- (pin-list:instrument = microcode)
- ((pin-list1:instrument, pin-list2:instrument) = microcode)

where:

pin-list	Is the pin name or pin group or (comma-separated list of pins) as specified in the instruments statement.
instrument	Is DCDiffMeter. This corresponds to the instrument type for pin-list as defined in the instruments statement.
microcode	A DCDiffMeter microcode.

The table lists the DCDiffMeter microcodes.

DCDiffMeter Microcodes	
Microcode	Description
Strobe	Strobes the meter and stores the measurement.
Enable_Inst_Cond	Enables the DCDiffMeter to drive the condition bit.

Disable_Inst_Cond	Disables the DCDiffMeter from driving the condition bit.
Enable_Alarm	Enables the alarm window.
Disable_Alarm	Disables the alarm window.
Clear_Cond	Clears the latched condition, while not disabling the condition bit from being driven by events on the DCDiffMeter.
Trig <name>	Starts a capture with the length and rate of the capture specified in the most recent PSet. See Trig Microcode.
Filter_Off	Turns the meter filter off.
Filter_On	Turns the meter filter on.
Enable_ASynch_Trig	Enables an asynchronous trigger.
Disable_ASynch_Trig	Disables an asynchronous trigger.
PSet <name>	Applies the PSet of the given name. See PSets.

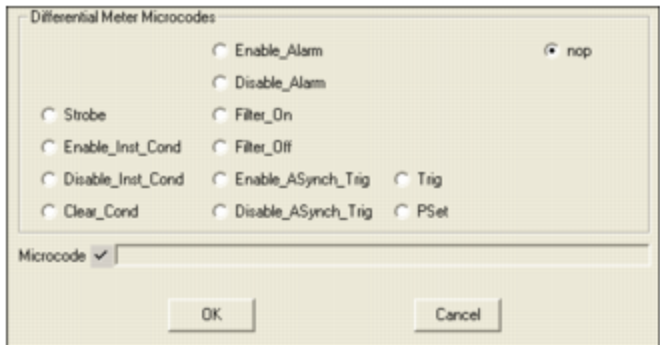
For more information, see:

- Instrument Microcode
- Pattern Example

DCDiffMeter PatternTool GUI

The PatternTool provides columns for the DCDiffMeter pins listed in the instrument statement of the compiled .atp file.

You can conveniently produce the comma-separated microcode equivalent by checking the check boxes as in shown in the figure.



DCDiffMeter Microcode Display

For more information, see:

- Pattern Example

- [Editing Instrument Microcode](#)

Trig Microcode

The [DCDiffMeter](#) accepts the Trig <name> syntax pattern. You assign names to signals in Mixed Signal workshop or using VBT. IG-XL ensures that all signals used by a pattern in SRM are loaded when the pattern is loaded into pattern memory. Each channel has memory for eight signal names. If you define and load more than eight signals per channel per pattern, IG-XL returns an error when loading the pattern.

[For more information, see Mixed Signal Workshop.](#)

<

>

2015 Teradyne, Inc. All rights reserved.

dcdiffmeter_program.3.17

<

>

DCDiffMeter Programming : DCDiffMeter Pattern Control : PSets

PSets

A PSet allows you to specify several aspects of an instrument’s state at once, such as the force voltage value and force voltage range. You program PSets using VBT code and apply them using either VBT or the PSet microcode. The DCDiffMeter supports PSet memory for a total 62 PSets per pattern. This includes any DCDiffMeter, DCVI, and DCTime PSets that may be stored in memory as well.

PSet properties for the DCDiffMeter are listed in the table below.

DCDiffMeter PSets	
DCDiffMeter PSet Property	Description
Capture	Returns an interface for controlling a DCDiffMeter’s capture capabilities.
ConditionBit	Controls the event that drives the condition bit. This property is only important for patterns.
VoltageRange	Programs the voltage range.

For more information, see TheHdw.DCDiffMeter.Pins.PSets.

PSet Programming Example

You program a PSet using PatternTool or a text editor. You define a PSet in VBT, as in the following example:

' Add the PSet to the PSet collection

TheHdw.DCDiffMeter.Pins("MyPin").PSets.Add "PSet1"

' Program DCDiffMeter instrument parameters for the PSet

TheHdw.DCDiffMeter.Pins("MyPin").PSets.Item("PSet1").ConditionBit = False

TheHdw.DCDiffMeter.Pins("MyPin").PSets.Item("PSet1").VoltageRange = 5

' Apply the programmed settings to the hardware

TheHdw.DCDiffMeter.Pins("MyPin").PSets("PSet1").Apply

You can also add a PSet to the collection as follows:

TheHdw.DCDiffMeter.Pins("MyPin").PSets.Item("MyPSet").Set 5, False, 1000 _
 , 300, 3

Applying a PSet

You apply a PSet from either a pattern or to the instrument in VBT. To apply a PSet with VBT, use the following syntax:

TheHdw.DCDiffMeter.Pins("MyPin").PSets.Item("MyPSet1").Apply

Calling a PSet from an ASCII pattern file uses the following format:

(MyPin:DCDiffMeter = PSet PSet1)

To call a PSet in PatternTool the setup would look as follows:

TSet	Command	Label	Vector/Cycle	Comment	ch1hi (DCDiffMeter)
-			0		
-			1		PSet PSet1
-			2		
-	repeat 500		3		
-			4		

PSet Called Using PatternTool

<

>

2015 Teradyne, Inc. All rights reserved.

dcdiffmeter_too.2.02

<

>

DCDiffMeter Theory of Operation : DCDiffMeter Overview

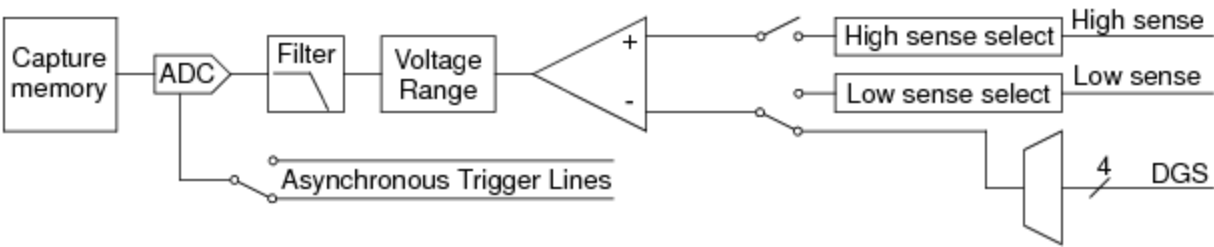
DCDiffMeter Overview

A DC Differential Voltmeter performs differential voltage measurements by connecting the high side to a DUT pin and the low side to a separate DUT pin or DGS.

The DCDiffMeter connects using a channel sense line for both the high and low side. The channel sense line that is used depends on what DCDiffMeter is being used and on other DUT connections. The low side can also connect to one of four DGS lines.

The DCDiffMeter has multiple filter settings that allow you to attenuate high frequency noise that may cause deviations in measurements.

The figure shows a simplified block diagram for a DCDiffMeter.



DCDiffMeter Block Diagram

For more information, see [DCDiffMeter Connections](#).

<

>

dcdiffmeter_too.2.03

<	>
-----------------	-----------------

[DCDiffMeter Theory of Operation](#) : DCDiffMeter Connections

DCDiffMeter Connections

DC30 and DC75 instrument boards have two DC Differential Voltmeters. Each DCDiffMeter connects to a certain set of sense channel lines. The following describes DCDiffMeter sense line connections:

- | | |
|---|---|
| • | DC30 DCDiffMeter |
| • | DC75 DCDiffMeter |
| • | DCDiffMeter Metering Ranges |
| • | Metering Filter |
| • | Waveform Capture |
| • | Asynchronous Trigger |

For more information, see [Connecting a DCDiffMeter](#).

<	>
-----------------	-----------------

--	--	--

dcdiffmeter_too.2.04



[DCDiffMeter Theory of Operation](#) : [DCDiffMeter Connections](#) : [DC30 DCDiffMeter](#)

DC30 DCDiffMeter

Each DC30 DCDiffMeter resides on one of two differential subbusses, allowing direct access to half of the channels and indirect access to the other half.

Each channel sense pin connects directly to one of the two differential subbusses. Measurements can be made between any two DUT pins on the board. Each DCDiffMeter can make a simultaneous measurement on any two channels located on the same subbus as the measuring DCDiffMeter.

If a DCDiffMeter is connected to a channel not on its differential subbus, only one DCDiffMeter measurement can be made at a time. The figure shows that this is due to the subbusses connecting and the need for the DCDiffMeters to share subbusses.

The DCDiffMeter that connects to the channels is based on the high side connection. If the high side is connected to a channel that DCDiffMeter 1 has a direct connection to, then DCDiffMeter 1 is used. If the high side connection is made to a channel that DCDiffMeter 2 has a direct connection to, then DCDiffMeter 2 is used.

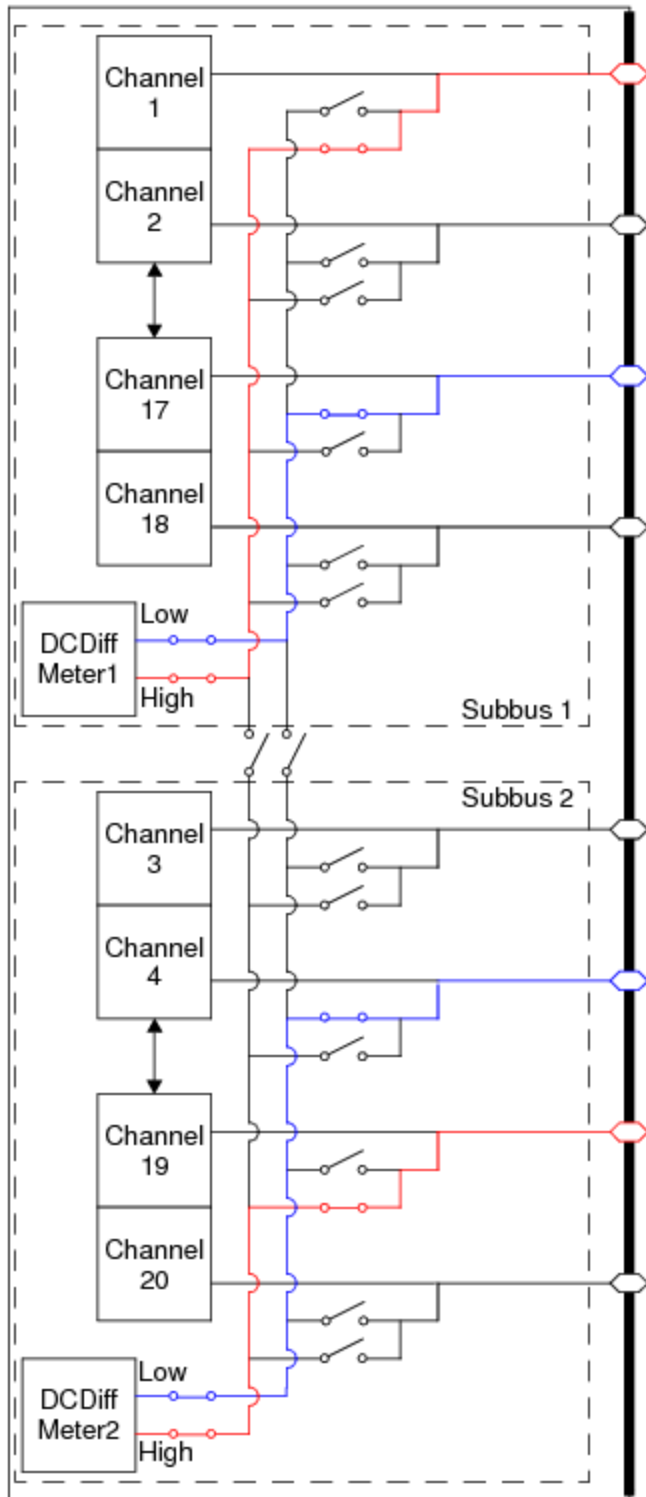
Note:

Choose the DIB layout carefully to take advantage of parallel differential measurements.

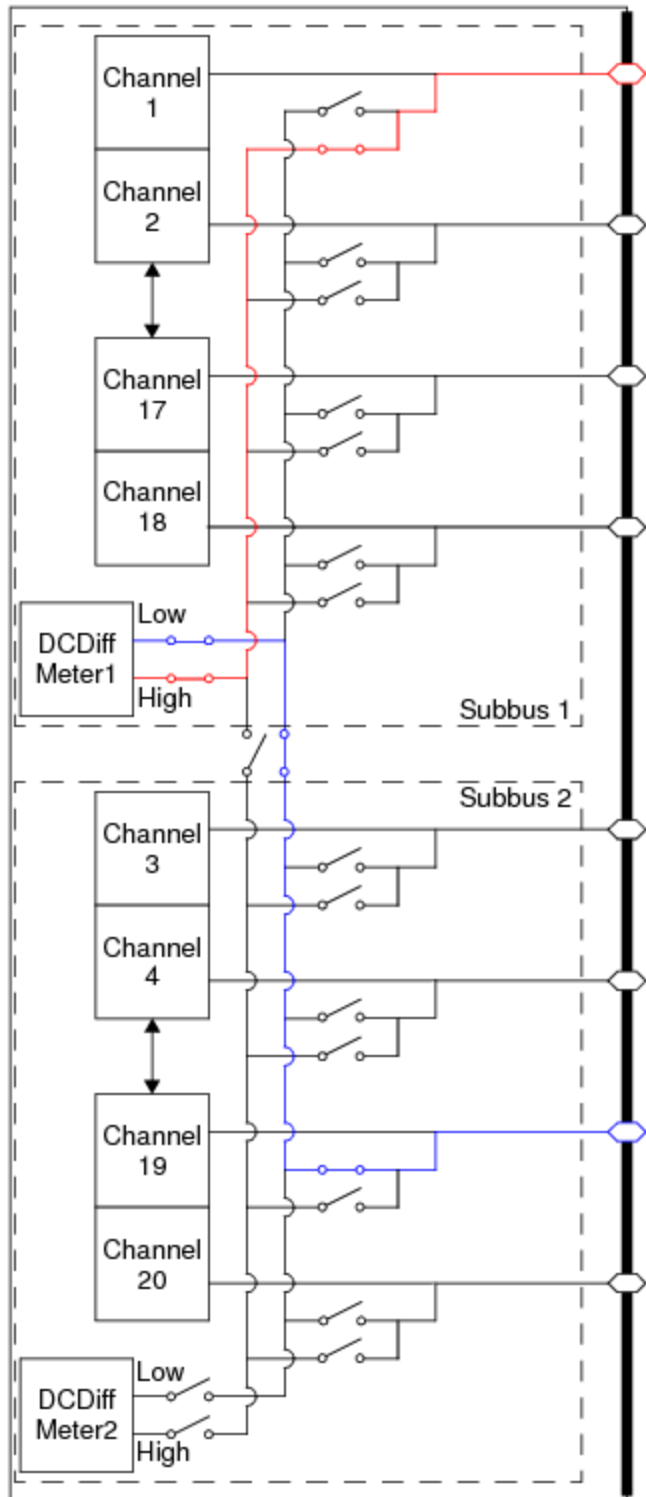
In the figure, the diagram on the left displays DCDiffMeter 1's high side connected to channel 1, and the low side is connected to channel 17 through subbus 1. On subbus 2, a second simultaneous differential meter measurement is being made. The DCDiffMeter 2's high side is connected to channel 19, and the low side is connected to channel 4.

The diagram on the right shows DCDiffMeter 1's high side connected to channel 1, while the low side is connected to channel 19 through the subbus. Since the DCDiffMeter is connecting through the subbus, only one DCDiffMeter measurement can be made at a time.

DCDiffMeter Connection Within Subbus



DCDiffMeter Connection Cross Subbus



DC30 DCDiffMeter Connections on the Subbus

For more information, see:

- [Connecting a DCDiffMeter](#)

- [DC30 Channel Connections](#)

- DCDiffMeter DGS

Potential Conflicts with Subbus Connections

Certain combinations of DC Differential Meter connections have the potential to create subbus conflicts, which can affect measurements. Consider the following example:

- Connect DCDiffMeter1 across the subbus (Hi to channel 1 and Lo to channel 4)
- Connect DCDiffMeter2 across the subbus (Hi to channel 3 and Lo to channel 2)

This combination has the potential to short channel 2 to channel 4 through the sbusLo1 to sbusLo2 connection, which may provide inaccurate measurements.

IG-XL 8.00.00 and later includes a check to determine if the subbus associated with a DCDiffMeter and with the low channel is already in use. If the low side bus is already in use, IG-XL returns the following error:

Cannot program the LowSide pin if the DCDiffMeter is already connected. Disconnect the currently connected DiffMeter pin(s) before changing the LowSide pin parameter.

To avoid getting this error, use the property `.LowSideCrossSBusConnectEnable` to disable low side subbus error messages.

For more information, see `LowSideCrossSBusConnectEnable` in the VBT Syntax Reference.

dcdiffmeter_too.2.05

<

>

DCDiffMeter Theory of Operation : DCDiffMeter Connections : DC75 DCDiffMeter

DC75 DCDiffMeter

Each DC75 DCDiffMeter high side connects directly to one of two channels. DCDiffMeter 1 connects to channels 1 or 2, while DCDiffmeter 2 connects to channels 3 or 4. The low side of each DCDiffMeter can connect to any of the four channels.

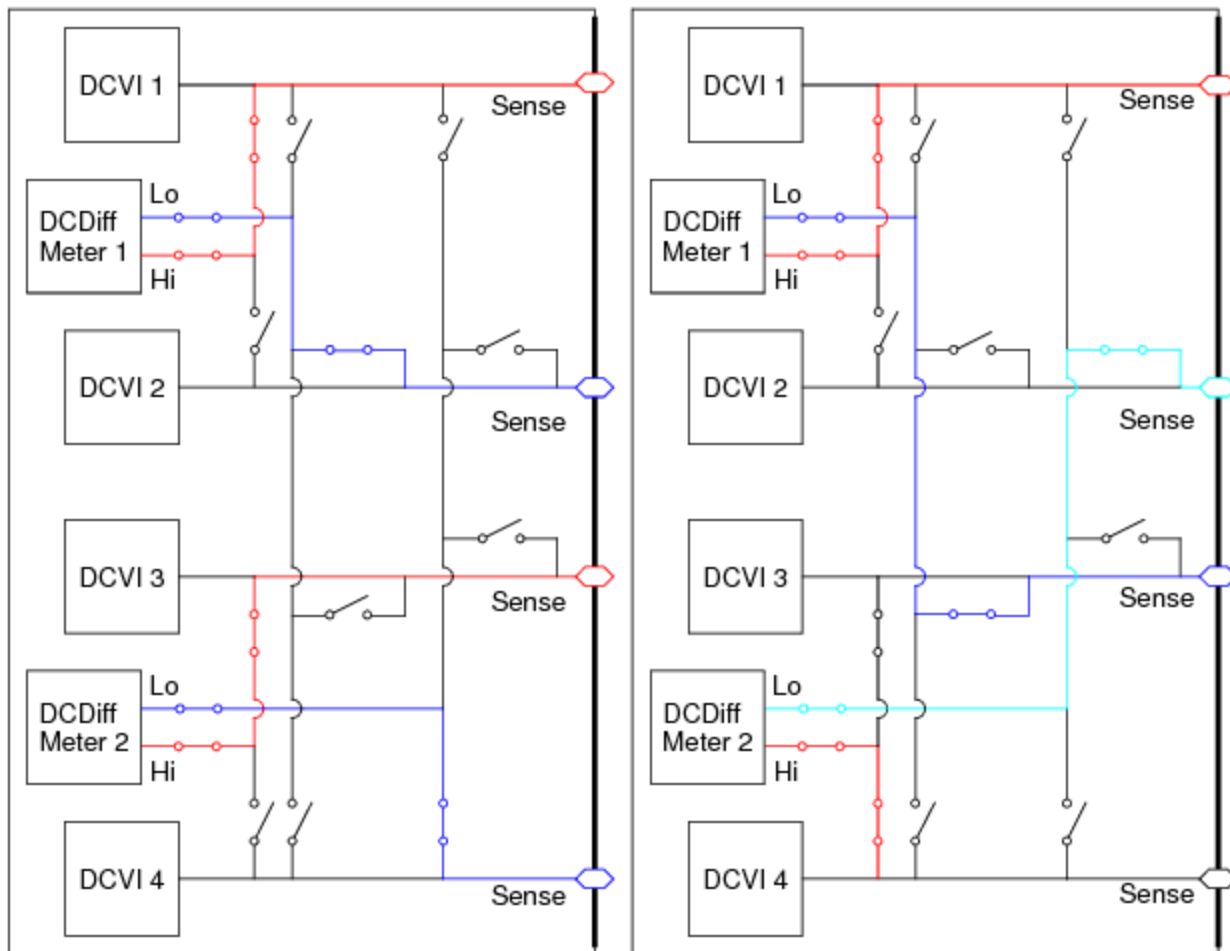
The table shows the Low side channel a DCDiffMeter can be attached to depending on the Hi side connected.

DC75 DCDiffMeter Connections			
DCDiffMeter 1		DCDiffMeter 2	
Hi Side	Low Side	Hi Side	Low Side
Chan 1	Chan 1, 2 ¹ , 3, or 4	Chan 3	Chan 1, 2, 3 ¹ or 4
Chan 2	Chan 1, 2 ¹ , 3, or 4	Chan 4	Chan 1, 2, 3 or 4 ¹

¹ Both inputs of either DCDiffMeter can be connected to same channel to perform spot calibration of the DCDiffMeter offset.

When laying out a DIB for the DCDiffMeter remember that the high side of each DCDiffMeter can only be connected to one of two channels.

This figure shows two possible connections the DCDiffMeter can make. The left side shows DCDiffMeter 1 high side connected to channel 1 and the low side connected to channel 2, while DCDiffMeter 2 high side is connected to channel 3 and the low side is connected to channel 4. In this setup the high side can be connected to one of two channels for each DCDiffMeter.



DC75 DCDiffMeter Setups

On the right side of the figure, DCDiffMeter 1 high side is connected to channel 1, while the low side is connected to channel 3. This would force DCDiffMeter to only be able to connect to channel 4, while the low side of the DCDiffMeter 2 could connect to channel 2.

For more information, see:

- [Connecting a DCDiffMeter](#)

- [DC75 Channel Connections](#)

- [DCDiffMeter DGS](#)



	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcdiffmeter_too.2.06

<	>
-----------------	-----------------

[DCDiffMeter Theory of Operation](#) : [DCDiffMeter Connections](#) : DCDiffMeter Metering Ranges

DCDiffMeter Metering Ranges

Each DCDiffMeter has ranges for voltage measurements. Ranges are provided to increase measurement accuracy.

DC30 and DC75 ranges are: 5V, 2.5V, 1V, 0.5V, 250mV, 100mV, and 50mV.

You can set ranges manually using VBT or PSets, or IG-XL can choose the best range using autoranging.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcdiffmeter_too.2.07

<	>
-----------------	-----------------

[DCDiffMeter Theory of Operation](#) : [DCDiffMeter Connections](#) : Metering Filter

Metering Filter

The DCDiffMeter has three filter settings that allow you to attenuate high frequency noise that may cause deviations in measurements.

The selectable bandwidths are: 500Hz, 10kHz, and Bypass. These filter bandwidths are typical values.

You can make a trade-off between settling time and measurement repeatability when selecting a meter filter. The lower the bandwidth the longer the settling time. The Bypass filter provides the least amount of filtering and the fastest settling time when making measurements.

Even with these available filter ranges, you can perform averaging and digital filtering of meter data.

For more information, see:

- [DC30 Specifications](#)

- [DC75 Specifications](#)

<	>
-----------------	-----------------

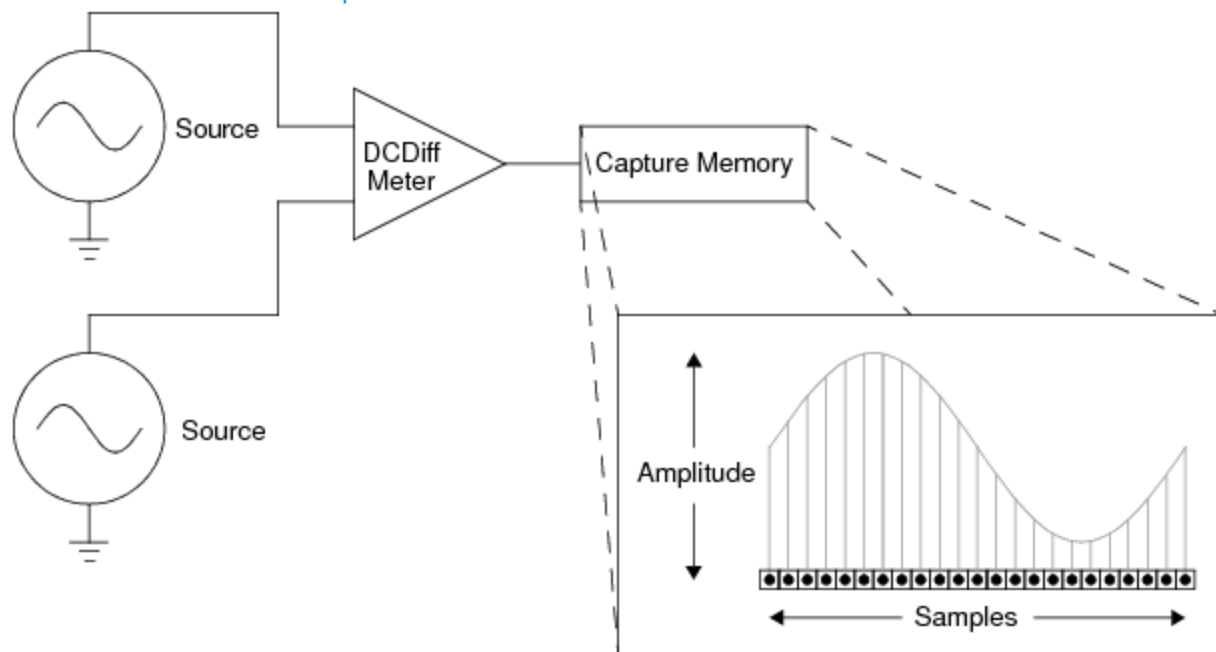
dcdiffmeter_too.2.08

DCDiffMeter Theory of Operation : DCDiffMeter Connections : Waveform Capture

Waveform Capture

Each DCDiffMeter can capture up to 512 samples of a waveform with a capture rates of 781.25Hz - 100kHz.

This figure shows a waveform being captured. The signal is converted to a series of points along the waveform and stored in capture memory. The points captured are based on the capture rate and the number of samples.



Waveform Capture Using DCDiffMeter

Each time the meter captures a value, it is stored in memory. A counter tracks how many values have been stored. If more than 512 samples are stored, the memory overwrites previously stored data and a data lost alarm is generated.

Capture Controls

You can trigger or stop a capture using VBT language, the DCVI debug display, or pattern microcode.

< >		
	2015 Teradyne, Inc. All rights reserved.	

dcdiffmeter_too.2.09

<	>
-----------------	-----------------

[DCDiffMeter Theory of Operation](#) : [DCDiffMeter Connections](#) : Asynchronous Trigger

Asynchronous Trigger

Measurements and captures can be initiated asynchronously by way of the DUT using the asynchronous trigger feature of the DCDiffmeter.

For more information, see:

- [Asynchronous Trigger](#)
- [Onboard Asynchronous Triggering of a DCDiffMeter](#)

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcdiffmeter_too.2.10

<	>
-----------------	-----------------

DCDiffMeter Theory of Operation : DCDiffMeter Alarms

DCDiffMeter Alarms

An alarm indicates that hardware is not operating as expected given the way an instrument is programmed. It indicates that a condition exists that could potentially invalidate a measurement. The DCDiffMeter alarms are:

- | | |
|---|---------------------|
| • | Capture |
| • | CaptureNotTriggered |
| • | DataLost |
| • | DGS |
| • | Overrange |
| • | Timeout |

<	>
-----------------	-----------------

dcdiffmeter_too.2.11

<	>
-----------------	-----------------

DCDiffMeter Theory of Operation : DCDiffMeter Alarms : Capture

Capture

An alarm occurred during a capture and the sample latched an alarm. Any of the DCDiffMeter alarms could have occurred while the capture was running.

Alarm Message

The following is the datalog alarm message:
{-1} : alarm [PinName, SiteNum] DDM : 0006 : Instrument alarmed during capture.
where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

Common Causes

- An alarm condition occurred during the capture and may cause a measurement to be invalid.
- Based on the alarm, the instrument may be in a different condition than expected.

<	>
-----------------	-----------------

dcdiffmeter_too.2.12

<	>
---	---

DCDiffMeter Theory of Operation : [DCDiffMeter Alarms](#) : CaptureNotTriggered

CaptureNotTriggered
Requested data on a sample was never triggered.
Alarm Message
The following is the datalog alarm message:
{-1} : alarm [PinName, SiteNum] DDM : 0009 : Capture not triggered.
where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

<	>
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcdiffmeter_too.2.13

<	>
-----------------	-----------------

DCDiffMeter Theory of Operation : DCDiffMeter Alarms : DataLost

DataLost

The capture overwrote the previous existing data due to a readback not occurring and no room was left in memory to retain all data.

Alarm Message

The following is the datalog alarm message:

{-1} : alarm [PinName, SiteNum] DDM : 0007 : Data lost. Memory was overwritten by subsequent captures.

where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcdiffmeter_too.2.14

<	>
---	---

DCDiffMeter Theory of Operation : DCDiffMeter Alarms : DGS

DGS

Identifies a difference in voltage between the DGS wire and DUT ground.

Alarm Message

The following is the datalog alarm message:

{-1} : alarm [PinName, SiteNum] DDM : 0004 : DGS alarm in the hardware.

where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

Common Causes

- Miswired DIB.

<	>
---	---

dcdiffmeter_too.2.15

<	>
---	---

DCDiffMeter Theory of Operation : DCDiffMeter Alarms : Overrange

Overrange

The input to the A/D converter in the voltmeter or ammeter is greater than the full-scale programmed range. The selected voltage or current range is too low.

Alarm Message

The following is the datalog alarm message:

{-1} : alarm [PinName, SiteNum] DDM : 0005 : Overrange alarm in the hardware.

where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

Common Causes

- | | |
|---|----------------------------|
| • | Incorrect range programmed |
| • | Defective device |

<	>
---	---

dcdiffmeter_too.2.16

<

>

DCDiffMeter Theory of Operation : DCDiffMeter Alarms : Timeout

Timeout

The capture data was not available when the instrument was trying to retrieve it.

Alarm Message

The following is the datalog alarm message:

{-1} : alarm [PinName, SiteNum] DDM : 0008 : Timeout occurred while retrieving data.

where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

Common Causes

- The data being retrieved took longer than the timeout setting.
- There is no data to retrieve.
- Two samples were requested but only one was taken. The hardware waits for the second sample for up to 100ms or the programmed timeout.

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcdiffmeter_too.2.17

<	>
-----------------	-----------------

DCDiffMeter Theory of Operation : DCDiffMeter Errors

DCDiffMeter Errors

An error is a condition that is determined by the hardware at run time and does not represent a risk of hardware damage. An error occurs when the hardware does not enter a programmed state or does not initiate a requested process.

The DCDiffMeter can return the following errors:

Missed PSET	Two PSets were executed too close together in time.
Missed Trigger	The VI meter was strobed while the A/D was still converting the previous reading. The conversion cycle is not restarted and the resulting value is the one from the previous strobe.
Wave Source Interrupt	A wave source is running and another is invoked before the first one ended.

<	>
-----------------	-----------------